

UNIT 4: Arrays in C

Introduction to Arrays

An **array** is a collection of elements of the same data type stored in contiguous memory locations. Arrays allow for efficient data management and easy access to multiple values using a single identifier.

Key Features of Arrays:

- **Fixed Size:** The size of an array is determined at the time of declaration and cannot be changed.
- **Homogeneous Elements:** All elements in an array must be of the same data type.
- **Indexing:** Elements are accessed using indices, starting from 0.
- **Contiguous Memory Allocation:** Arrays store elements in contiguous memory locations, making access and traversal faster.
- **Random Access:** Arrays provide constant-time access to elements using their indices.

Why Arrays are needed:

1. **Efficient Data Storage:** Arrays allow storing multiple values of the same type under a single variable name.
2. **Easy Access:** Using indices, any element in the array can be directly accessed.
3. **Better Memory Utilization:** Arrays make it easier to allocate and manage memory for a fixed number of elements.
4. **Simplifies Code:** Instead of declaring multiple variables, arrays simplify the process by grouping related data together.
5. **Facilitates Iteration:** Arrays can be traversed using loops, making them ideal for operations on large data sets.

Syntax:

```
data_type array_name [size];
```

Example:

```
int numbers[5]; // Declares an array of 5 integers.
```

Declaration, Initialization, and Accessing Elements of Arrays:

1. Declaring Arrays:

Arrays can be declared by specifying their data type, name, and size.

Syntax:

```
data_type array_name [size];
```

Example:

```
int numbers[5]; // Declares an integer array of size 5.
```

```
char letters[10]; // Declares a character array of size 10.
```

```
float prices[3]; // Declares a float array of size 3.
```

2. Initializing Arrays:

Arrays can be initialized at the time of declaration or after declaration.

a. During Declaration/Compile Time Initialization:

You can assign values to the array elements while declaring the array.

```
int numbers[5] = {1, 2, 3, 4, 5};
```

Partial Initialization: If fewer values are provided, the remaining elements are initialized to 0 (for numeric types).

```
int numbers[5] = {1, 2}; // Remaining elements are set to 0.
```

b. Dynamic Initialization/Run time Initialization:

Values can be assigned during runtime using loops.

```
int numbers[5];
for (int i = 0; i < 5; i++) {
    numbers[i] = i * 10; // Assigning values dynamically.
}
```

3. Accessing Array Elements:

Elements of an array are accessed using their index. The index starts from 0.

Syntax:

```
array_name [index];
```

Example:

```
int firstElement = numbers[0]; // Accessing the first element.
```

```
Numbers [2] = 25; // Modifying the third element.
```

Q. Write a program in C to store elements in an array and print them.

```
#include <stdio.h>
int main(){
    int arr[10],i; // Declare an array of size 10 to store integer values

    // Prompt the user to input 10 elements into the array
    printf ("Input 10 elements in the array :\n");

    for (i=0; i<10; i++){
        printf("element - %d : ",i); // Prompt the user to input the ith element
        scanf("%d", &arr[i]); // Read the input and store it in the array
    }
```

```
// Display the elements in the array
printf("\n Elements in array are: ");
for (i=0; i<10; i++){
    printf("%d\n ", arr[i]); // Print each element in the array
}
return 0;}
```

Q. Write a program in C to read n number of values in an array and display them in reverse order.

Q. Write a program in C to find the sum of all elements of the array.

```
#include <stdio.h>
int main(){
    int a[100]; // Declare an array of size 100 to store integer values
    int i,n,sum =0; // Declare variables to store array size, loop counter and sum

    // Prompt the user to input the number of elements to be stored in the array
    printf("Input the number of elements to be stored in the array :");
    scanf("%d", &n);

    // Prompt the user to input n elements into the array
    printf("Input %d elements in the array :\n", n);
    for(i = 0; i < n; i++){
        printf("element - %d : ", i);
        scanf("%d", &a[i]); // Read the input and store it in the array
    }

    // calculate the sum of all elements in the array using a for loop
    for(i = 0; i < n; i++){
        sum += a[i]; // Add each element to the sum
    }

    // Display the sum of all elements stored in the array
    printf("Sum of all elements stored in the array is : %d\n\n", sum);
    return 0;
}
```

Q. Write a program in C to copy the elements of one array into another array.

Q. Write a program in C to sort elements of an array in ascending order.

```
#include <stdio.h>
void main()
{
    int arr1[100],n, i, j, tmp;

    // Prompt user for input
    printf("Input the size of array : ");
    scanf("%d", &n);

    // Input elements for the array
    printf("Input %d elements in the array :\n", n);
    for (i = 0; i < n; i++){
        printf("element - %d : ", i);
        scanf("%d", &arr1[i]);}

    // sorting elements in ascending order using the Bubble Sort algorithm
    for (i = 0; i < n; i++){
        for (j = i + 1; j < n; j++){
            if (arr1[j] < arr1[i]){
                // Swap elements if they are in the wrong order
                tmp = arr1[i];
                arr1[i] = arr1[j];
                arr1[j] = tmp;
            }
        }
    }

    // Print sorted elements in ascending order
    printf("\nElements of array in sorted ascending order:\n");
    for (i = 0; i < n; i++){
        printf("%d\n ", arr1[i]);}
}
```

Q. Write a program in C to find the maximum/Largest and minimum/smallest elements in an array.

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int arr1[100];
```

```
    int i, mx, mn, n;
```

```
// Prompt user for input
```

```
    printf("Input the number of elements to be stored in the array :");
```

```
    scanf("%d", &n);
```

```
// Input elements for the array
```

```
    printf("Input %d elements in the array :\n", n);
```

```
    for (i = 0; i < n; i++){
```

```
        printf("element - %d : ", i);
```

```
        scanf("%d", &arr1[i]);
```

```
    }
```

```
// Initialize max (mx) and min (mn) with the first element of the array
```

```
mx = arr1[0];
```

```
mn = arr1[0];
```

```
// Traverse the array to find the maximum and minimum elements
```

```
for (i = 1; i < n; i++){
```

```
    // Update mx if the current element is greater
```

```
    if (arr1[i] > mx){
```

```
        mx = arr1[i];}
```

```
    // Update mn if the current element is smaller
```

```
    if (arr1[i] < mn){
```

```
        mn = arr1[i];}
```

```
}
```

```
// Print the maximum and minimum elements
```

```
printf("Maximum element is : %d\n", mx);
```

```
printf("Minimum element is : %d\n\n", mn);
```

```
    return 0;
```

```
}
```

Multi-Dimensional Array

A **multi-dimensional array** is an array of arrays. Essentially, it's an array that contains other arrays as its elements, which allows it to store data in more than one dimension.

The most common type is the **two-dimensional array**, which is often used to represent matrices or tables. However, you can have arrays with three or more dimensions.

Types of Multi-Dimensional Arrays:

- **Two-Dimensional Array:** Often used to represent tables or grids (rows and columns).
- **Higher-Dimensional Arrays:** Can have four or more dimensions, typically used in complex scientific computing.

Declaration of Multi-Dimensional Arrays:

In most programming languages, the syntax for declaring a multi-dimensional array involves specifying the dimensions and size.

1. Two-Dimensional Array Declaration:

In C, C++, Java, etc., you can declare a two-dimensional array like this:

```
datatype array_name[row_size][column_size];
```

- `datatype` is the type of data the array will store (e.g., `int`, `float`).
- `array_name` is the name of the array.
- `row_size` is the number of rows in the array.
- `column_size` is the number of columns in the array.

Example:

```
int matrix[3][4]; // Declares a 2D array with 3 rows and 4 columns.
```

Initialization of Multi-Dimensional Arrays:

There are several ways to initialize a multi-dimensional array, including:

1. Static Initialization (Using Values)

You can initialize a multi-dimensional array when declaring it by providing the values within braces `{}`.

Example:

```
int matrix[2][3] = {{1, 2, 3},{4, 5, 6}};
```

Here, the array `matrix` has 2 rows and 3 columns, and the values are initialized directly.

2. Dynamic Initialization (Using Loops)

You can also use loops to initialize the elements of a multi-dimensional array.

Example:

```
int matrix[2][3];
for (int i = 0; i < 2; i++) {
    for (int j = 0; j < 3; j++) {
        matrix[i][j] = (i + 1) * (j + 1); // Assign values dynamically
    }
}
```

Accessing Elements of Multi-Dimensional Arrays

To access elements in a multi-dimensional array, you need to specify the index of each dimension.

1. Accessing Elements in Two-Dimensional Arrays:

In a 2D array, the element is accessed using two indices: one for the row and one for the column.

Example:

```
int matrix[2][3] = { {1, 2, 3}, {4, 5, 6} };
int value = matrix[1][2]; // Accessing element in the 2nd row, 3rd column (5th element)
printf("%d", value); // Output: 6
Here: matrix[1][2] accesses the element in the second row (index 1) and third column (index 2).
```

Q. Write a program in C for a 2D array of size 3x3 and print the matrix

```
#include <stdio.h>
```

```
void main() {
    int arr1[3][3], i, j;

    // Input values for the matrix
    printf("Input elements in the matrix :\n");
    for (i = 0; i < 3; i++) {
        for (j = 0; j < 3; j++) {
            printf("element - [%d],[%d] : ", i, j);
            scanf("%d", &arr1[i][j]); } }
}
```

```
// Display the matrix
printf("\nThe matrix is : \n");
for (i = 0; i < 3; i++) {
    printf("\n");
    for (j = 0; j < 3; j++)
```

```
printf("%d\t", arr1[i][j]); } }
```

Q. Write a program in C for adding two matrices of the same size.

```
#include <stdio.h>
```

```
void main() {  
    int arr1[50][50], brr1[50][50], crr1[50][50], i, j, n;  
  
    // Prompt user for input  
    printf("Input the size of the square matrix (less than 5): ");  
    scanf("%d", &n);  
  
    // Input elements for the first matrix  
    printf("Input elements in the first matrix :\n");  
    for (i = 0; i < n; i++) {  
        for (j = 0; j < n; j++) {  
            printf("element - [%d],[%d] : ", i, j);  
            scanf("%d", &arr1[i][j]); } }  
  
    // Input elements for the second matrix  
    printf("Input elements in the second matrix :\n");  
    for (i = 0; i < n; i++) {  
        for (j = 0; j < n; j++) {  
            printf("element - [%d],[%d] : ", i, j);  
            scanf("%d", &brr1[i][j]); } }  
  
    // Calculate the sum of the matrices  
    for (i = 0; i < n; i++){  
        for (j = 0; j < n; j++){  
            crr1[i][j] = arr1[i][j] + brr1[i][j]; } }  
  
    // Display the addition of two matrices  
    printf("\nThe Addition of two matrix is : \n");  
    for (i = 0; i < n; i++) {  
        printf("\n");  
        for (j = 0; j < n; j++)  
            printf("%d\t", crr1[i][j]);  
    }  
}
```

Q. Write a program in C to calculate and display the sum of all elements of a given matrix.

Q. Write a program in C to calculate and display the sum of odd elements of a given matrix.

Q. Write a program in C to calculate and display the sum of even elements of a given matrix.

Q. Write a program in C to find the transpose of a given matrix.

```
#include <stdio.h>
```

```
int main() {
    int arr1[50][50], brr1[50][50], i, j, r, c;

    // Input the rows and columns of the matrix
    printf("\nInput the rows and columns of the matrix: ");
    scanf("%d %d", &r, &c);

    // Input elements in the first matrix
    printf("Input elements in the matrix:\n");
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) {
            printf("element - [%d],[%d] : ", i, j);
            scanf("%d", &arr1[i][j]); } }

    // Transpose of the matrix
    for (i = 0; i < r; i++) {
        for (j = 0; j < c; j++) {
            // Assigning transposed values to the new matrix
            brr1[j][i] = arr1[i][j]; } }

    // Display the transpose of the matrix
    printf("\n\nThe transpose of a matrix is : ");
    for (i = 0; i < c; i++) {
        printf("\n");
        for (j = 0; j < r; j++) {
            printf("%d\t", brr1[i][j]); } }
    return 0;
```

Q. Write a program in C for the multiplication of two matrices.

```
crr1[i][j] = sum; } }
```

```
// Display the result of matrix multiplication
```

```
printf("\nThe multiplication of two matrices is:\n");
```

```
for (i = 0; i < r1; i++) {
```

```
    printf("\n");
```

```
    for (j = 0; j < c2; j++)
```

```
        printf("%d\t", crr1[i][j]);
```

```
    }
```

```
}
```

```
return 0;
```

```
}
```